SMETRIC: Rethink LLM Scheduling for Serving Agents with Balanced Session-centric Scheduling

Jiahao Wang[1], Kaizhan Lin[†1], Kaixi Zhang[1], Jinbo Han[1], Xingda Wei[✉1], Sijie Shen[2], Chenguang Fang[2], Wenyuan Yu[2], Rong Chen[1], Haibo Chen[1]

[1]Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University [2]Alibaba Group

arXiv:2607.08565v2 [cs.DC] 13 Sep 2026

Abstract

LLM scheduling is critical to serving, yet how well existing designs fit agentic serving—where agents, not humans, issue the requests—remains unclear. Agents shift the workload in two ways: they consume many more tokens than humans, so the cluster must provide high throughput (tokens per second, TPS) at low latency; and their requests reuse far more KV$ than chat.

Existing schedulers still trade off load balance against KV$ reuse under the new workloads: cache-aware schedulers may crowd requests onto the few instances caching the KV$, leaving the rest idle, while balanced schedulers may lose the opportunity for reuse, which is costly, as the benefit of reuse becomes more dominant at a high reuse ratio. We thus present two key insights: (1) with a global-tier KV$ store, pursuing load balance need not compromise KV$ reuse, though the slower global tier must be used with care; and (2) given the agent's intra-session locality, routing requests by their sessions can balance the load with high KV$ reuse.

A key challenge in realizing session-centric scheduling is that the scheduler must identify a request's session statelessly, which is difficult for model providers serving arbitrary agents. SMETRIC addresses this with differential scheduling based on two indicators derived from the request itself, the *session turn* and the local KV$ hit: it schedules first-turn requests for load balance, as they mark the arrival of new sessions, and sticks follow-ups to the instance with the highest local hit for high KV$ reuse. As sessions differ widely in size, SMETRIC sticks a follow-up only if the instance can serve it within its SLO, and otherwise migrates the session to the least-loaded instance to prevent many long sessions from eventually imbalancing the load. Evaluated on real-world traces, SMETRIC improves the peak TPS by 9–15 % under prefill-decode colocation with a provisioned global tier and the peak prefill TPS by 9 % under disaggregation over state-of-the-art schedulers, also with lower latency.

[†]Kaizhan Lin is affiliated with ShanghaiTech University; this work was done while he was an intern at Institute of Parallel and Distributed Systems, Shanghai Jiao Tong University.

[✉]Xingda Wei is the corresponding author (wxdwfc@sjtu.edu.cn).

1 Introduction

Large language models (LLMs) now power agents like Claude Code [6, 48] that accomplish complex tasks such as writing [5], debugging [16], and performance-tuning [54, 42] code. An agent solves a task through a *session* of LLM requests, each of which requires generating many tokens. To generate these tokens efficiently, providers deploy clusters dedicated to serving agent requests; each cluster runs many instances, each holding a full copy of the model. We term this cluster-scale, agent-dedicated LLM serving *agentic serving*. Agentic serving is still LLM serving: the provider serves each request through the standard LLM API and neither runs the agents nor sees their internal logic.

Request scheduling—how the cluster routes each request to an instance—is a key pillar of LLM serving. Though recent work studies it extensively [75, 82, 59, 74, 37, 2], we argue that scheduling designs need to be revisited for agents, for two reasons:

First, an agent consumes far more tokens per request than a human user [9], so a cluster must provide high throughput (tokens per second, TPS) to serve the load at low cost. Besides cost, the latency—the time to first token (TTFT) and the time per output token (TPOT) [34]—cannot be compromised either. This is because the token generation latency determines how fast agents finish their tasks, and a response that is too slow makes the agent treat the request as failed [8].

Second, agentic serving exhibits a different workload pattern from chat: KV$ reuse dominates. The efficiency of serving an LLM request depends heavily on KV$ reuse; the KV$ is the intermediate state a cluster caches to avoid recomputation (§2). In chat workloads, this reuse is modest: only 54–62 % of request tokens are reused from cache [63]. In agentic serving, it is far higher: our analysis of the production trace from a cluster at a representative model provider (BAILIAN) reveals that the reuse exceeds 80 % (Finding 1 in §3). This shift requires revisiting how well scheduling designs facilitate KV$ reuse, because the computation saved by reuse becomes more dominant at a high reuse ratio: raising it from 70 % to 80 % cuts the prompt tokens still to compute by one third.

In this paper, we first conduct a systematic study of request scheduling for agents, using two recent large-scale real-world agentic serving traces (§3). We find that state-of-the-art

schedulers still suffer from the trade-off between KV$ reuse and load balance [75]. Specifically, to achieve high reuse, the current scheduler in BAILIAN prioritizes routing each request to the instance holding its cached KV$ (cache-aware). This trades away load balance: some instances overload while others sit idle, capping the cluster's throughput. On the other hand, a balanced scheduler like LMetric [75] trades away KV$ reuse, requiring more computation to serve the requests, so the throughput gain from better load balancing is limited.

To address this dilemma, we draw two insights. First, from the system's perspective, pursuing load balance does not have to compromise KV$ reuse: modern agentic serving adopts a two-tier KV$ store—a local tier on each instance's GPU memory and a global tier on cluster-wide CPU memory (§2). Even if the scheduled instance does not hold the KV$ locally, it can fetch it from the global tier. As long as the global tier serves these KV$ fetches in time, TPS remains high (§4.1). The global tier, however, must be used with care: it is slower—waiting for the KV$ to be transferred adds latency—and its capacity and bandwidth are limited. Routing *every* request purely for balance—even with global-tier KV$ reuse—is thus still suboptimal.

Second, from the agent workload's perspective, routing requests by their *sessions* can balance the load with high KV$ reuse. On the one hand, sessions are a natural unit for balancing: though individual sessions have skewed token usage, the token load *can* still be balanced across instances at scale (Finding 6). On the other hand, sessions capture most of the reuse, thanks to the workload's intra-session locality: follow-up requests mostly reuse the KV$ of earlier turns in the same session (Finding 2). Keeping each session's requests on one instance thus preserves most of the KV$ reuse in its local tier, while spreading sessions across instances balances the cluster. At first glance, the KV$ reuse of first-turn requests seems compromised. The impact is small: their reuse comes mostly from the system prompt (Finding 3), which the global tier serves efficiently.

One key challenge in realizing such session-centric scheduling is that the scheduler must identify each request's session *statelessly*, which is difficult for model providers serving arbitrary agents: the request carries no session information, as the LLM APIs are stateless, and its source may further be anonymized by proxies. Inferring sessions from records of past requests is typically impractical, as such per-session state grows with active sessions, making a simple and scalable router design challenging.

We address this challenge with SMETRIC. The key observation is that the *session turn* and the local KV$ hit, both derived from the request itself, identify sessions statelessly and at zero cost. The turn, deduced from the number of historical messages in the request, tells the scheduler whether the request starts a session: SMETRIC thus schedules first-turn requests for load balance, spreading sessions across the cluster (§4). Meanwhile, the local KV$ hit, which the router already knows, helps identify the instance serving the session, because intra-session reuse happens quickly (Finding 4), so the instance with the highest hit is likely the one that executed the session's previous request. Therefore, SMETRIC sticks follow-up requests to it. This differential scheduling achieves load balance and high local-tier reuse while keeping demand on the global tier low, because in the ideal case only first-turn requests fetch KV$ from it.

While such differential scheduling identifies sessions statelessly, we meet a second challenge: sticking every follow-up request to its session's instance still imbalances the load, because sessions differ widely in size (§3.2) and the size is unknown when the session is first routed, so an instance overloads as multiple long sessions are scheduled onto it. SMETRIC thus sticks a follow-up request only if the instance can finish it within its TTFT SLO, and otherwise migrates the session by routing the request to the least-loaded instance, with the global tier preserving its KV$ reuse (§4).

We have implemented SMETRIC on vLLM [61] and LMCache [38], and extensively evaluated it against state-of-the-art schedulers, including both the production scheduler used in BAILIAN and recent research and open-source schedulers (LMetric [75], llm-d [37], Preble [59], and Dynamo [45]), with serving paradigms spanning both prefill-decode colocation and prefill-decode disaggregation. Against the best-performing baseline, SMETRIC serves 9–15 % more peak TPS under colocation with a provisioned global tier, and 9 % more peak prefill TPS under disaggregation; the gap widens to 26–28 % as the load grows. It also achieves the lowest median TTFT and the lowest P90 TPOT under colocation, and the lowest tail TTFT under disaggregation (19 % lower at P90 than the closest baseline).

In summary, this paper makes the following contributions:

- The first study of the workload patterns of LLM requests served by an agent-dedicated cluster at scale (§3).
- The first analysis of how existing schedulers perform with the two-tier KV$ store under agentic serving (§4.1).
- A session-centric scheduling design that identifies sessions statelessly by the turn and the local KV$ hit, and migrates overloaded sessions by their SLO, achieving both high KV$ reuse and load-balanced serving (§4.2).
- An evaluation showing that SMETRIC improves TPS and latency over state-of-the-art schedulers (§5).

We will open-source SMETRIC and the traces upon publication.

2 Background and System Setup

LLM and KV$. An LLM serves a request in two phases. Given the input tokens (the *prompt*), a *prefill* pass processes the prompt with a single model forward (possibly split into chunks [1]) and emits the first output token. A *decode* pass

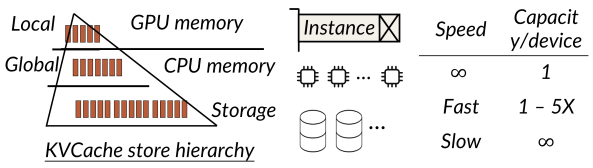


Figure 1: The KV$ store hierarchy for serving agents.

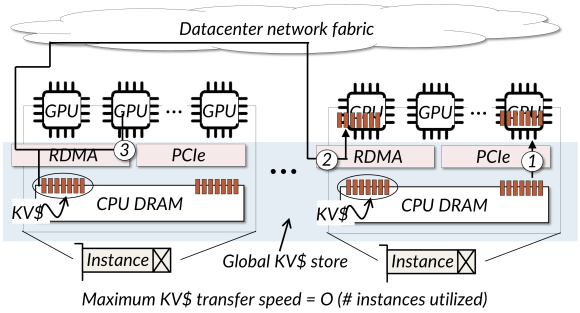


Figure 2: The detailed architecture of global-tier KV$. The global tier enables a GPU to read KV$ from local CPU (①), remote CPU memory (②) and remote GPU memory via GPU-direct RDMA (③).

then emits the remaining tokens auto-regressively: each token comes from one model forward over the prompt plus all tokens emitted so far, until the model produces an end-of-sequence (EOS) token.

The forward pass of LLMs requires computing a key (K) and a value (V) tensor for each token in the input sequence. Since these tensors are the same for inputs with the same prefix, it is standard to cache the KV tensors generated so far (the KV$) to accelerate the decode phase, as well as the prefill phase of later requests with the same prefix. As shown in Figure 4 (a), if a 3-token input request has two KV$ hits, then it needs to compute the KV$ for only one token.

KV$ store hierarchy and the local vs. global tiers. Since cross-request reuse is common in production workloads [63, 53], serving systems typically cache the KV$ of finished requests for some time. Caching requires storage capacity because each request's KV$ is large: for example, a single 128K-token request consumes about 32 GB of KV$ on the popular Qwen3-32B model. As a result, the KV$ store in production systems follows a multi-tier architecture as shown in Figure 1. The first tier is device (GPU) memory; when it fills, KV$ cascades to the larger, global tiers below—host (CPU) memory, then cloud storage (e.g., S3). A later KV$ hit on a lower tier must load the entry back up to the GPU.

Agentic workflow and the agent session. Modern LLM agents such as Claude Code [6] solve a task through a *session* of multiple rounds of interaction with the LLM. As shown in Figure 3, the first request (Request #1) in a session, typically written by the user, specifies the task to solve. The LLM replies with tokens that carry its reasoning and the next step for the agent, such as a tool call that reads a file with `cat`

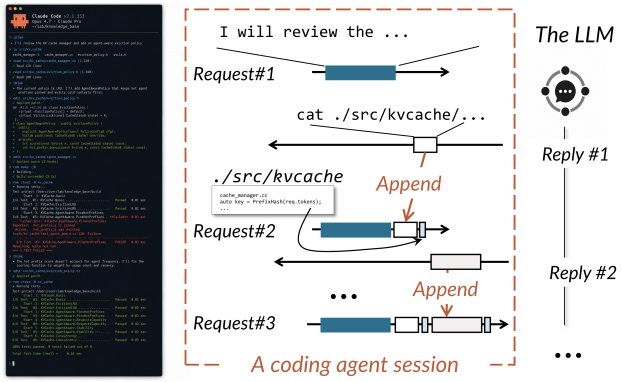


Figure 3: An example of how agents interact with the served LLM.

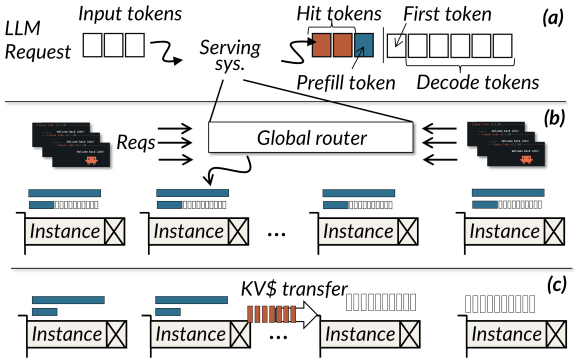


Figure 4: (a) An overview of the prefill and decode of LLM processing with KV$. (b) The PD-colocated and (c) the PD-disaggregated serving architecture.

(Reply #1). The agent then runs the tool, appends the tool call output to the context, and issues the next request (Request #2). Because the LLM keeps no state across requests, each request resends the whole history. This append-only pattern—request i extends request $i-1$—lets request i hit the KV$ of request $i-1$, if the platform has not evicted it.

LLM serving architecture in the wild. To serve LLM requests at scale, providers typically deploy each model on a cluster of GPUs[1]. The smallest unit of deployment is a *model instance*—one or more GPUs that together hold a full copy of the model parameters. A *global router* dispatches each incoming request to one instance using a routing algorithm [60, 59, 82, 75, 65]. Because a request runs in two phases—prefill and decode—there are two mainstream serving architectures today:

PD colocation. PD colocation runs the prefill and decode of a request to completion on the same instance (Figure 4 (b)).

PD disaggregation. PD disaggregation uses two clusters for serving: one cluster only runs prefill and the other only runs decode [80, 50] (Figure 4 (c)). A request first runs prefill

[1]Providers may deploy multiple clusters; inter-cluster scheduling is beyond the scope of this work.

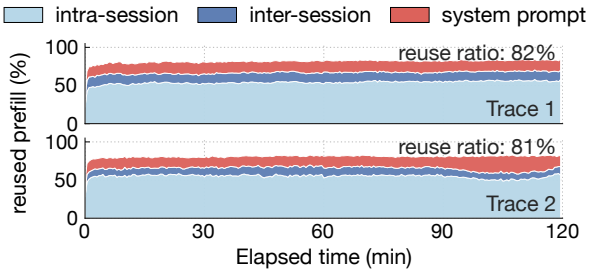


Figure 5: A characterization of the reusable KV$ for all requests, with a breakdown of where the reuses come from.

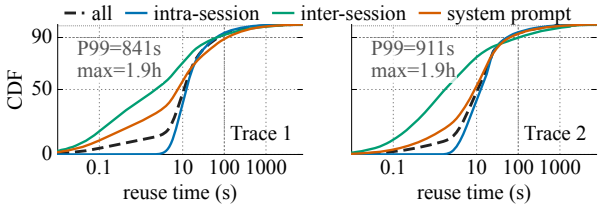


Figure 6: Reuse-time distribution across the two production traces. The tail is truncated by the 2-hour trace collection window.

on a prefill instance, which transfers the resulting KV$ over RDMA to a decode instance, where the decode phase runs to completion.

Both architectures build atop the same KV$ hierarchy of Figure 1 and differ only in which instances access the global tiers. Since cross-request KV$ reuse happens at prefill, every instance accesses the global tiers under PD colocation, while only prefill instances do under PD disaggregation.

3 Agentic serving patterns in the wild

Analyzed traces. Existing production workload studies characterize LLM serving in general [68, 53]. To understand the workload characteristics of real-world agentic serving, we collected two recent representative traces from two large-scale clusters, both dedicated to serving agents for a commercial coding-agent product (a paid subscription plan). Each cluster is one of many supporting the product and contains at least thousands of accelerators. Both traces cover the same peak usage hours (10:00–12:00) on May 29, 2026: one cluster serves a popular TB-level model (Trace 1) and the other a roughly 280B-parameter model (Trace 2), covering agents with different capabilities.

To study the workload in depth and to evaluate the serving system under it, we collected as much information as possible within BAILIAN's privacy policy. For each LLM request, each trace records (1) the hashed request content, which lets us analyze how much the request can benefit from KV$ reuse, (2) the arrival time, which captures request burstiness and KV$ reuse intervals, and (3) the session information—the session ID, the parent request ID, the turn number, and how the request is triggered (e.g., by a human or by the agent). Since requests carry no session identifier, we reconstruct the session ID and the parent request ID offline during trace collection (§A).

3.1 KV$ reuse patterns

We analyze three metrics of KV$ usage in agentic serving—the reuse ratio, where the reuse comes from, and the reuse interval—and distill four findings:

Finding 1: KV$ reuse is dominant: the reuse ratio exceeds 80 % of the KV$ in both traces (Figure 5).

We measure reuse at KV$ block granularity against an idealized KV$ store with infinite capacity, using the block size of the traced cluster. A request's reusable ratio is the fraction of its prefill tokens that fall in hit blocks, i.e., the tokens whose prefill computation can be skipped.

Finding 2: KV$ reuse follows session locality: more than 65 % of reuses come from *follow-up* requests (the second turn onward) reusing earlier turns of the same session (Figure 5).

For each reuse, we track which session originally produced the reused tokens. Figure 5 further breaks down the reuse on this basis into intra-session reuse, when a request's reused part comes from a previous request in the same session, and inter-session reuse, when it comes from a request in a different session. Intra-session reuse contributes most of the reuse (roughly 67 % in both traces), which is unsurprising: different sessions tend to solve different problems, so cross-session reuse is rare. Much of the remaining inter-session reuse stems from multi-session workflows [4, 44], in which an agent forks multiple sub-agents whose sessions share the parent session's context. Such attribution by the original producer also makes 67 % a lower bound on session locality: when a sub-agent's follow-up request reuses its own earlier turns, we count the reuse as inter-session, though it could also be counted as intra-session.

Finding 3: First-turn requests enjoy KV$ reuse: the shared system prompt accounts for 18–20 % of all reuses (Figure 5).

A first-turn request has no earlier turn in its session to reuse, yet it still hits on the system prompt: requests from the same agent framework, e.g., Claude Code [6] or OpenClaw [48], share one prompt template for the first request of each session. This is also a case of inter-session reuse.

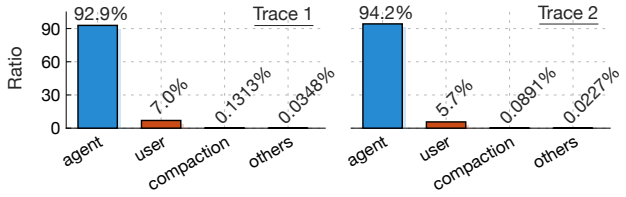


Figure 7: A breakdown of what triggers the requests in the two traces. *Agent* denotes requests automatically sent by the agent; *user* denotes requests sent by the human; *compaction* is a special agent-triggered request that summarizes the context of the current session.

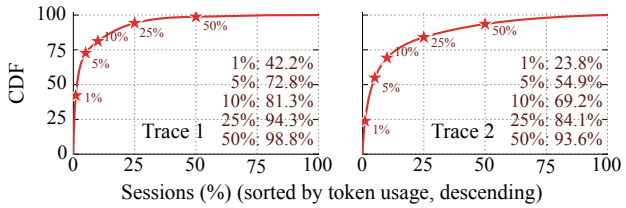


Figure 8: An analysis of token usage of different sessions.

Finding 4: KV$ is quickly reused: about 90 % of all reuses recur within roughly 100 s after the last request that shares the KV$, with only a small tail (Figure 6).

Figure 6 plots the reuse interval—the time from when a request completes until its KV$ is next reused. Reuse is quick, often within seconds, because both traces are dominated by agent-triggered requests (Figure 7): the agent issues the next request as soon as it sees the previous response and tool results. A few outliers are reused hours later, likely when a user leaves the agent waiting for a response.

3.2 Session execution patterns

Next we examine the detailed pattern of each session and derive the following two key findings:

Finding 5: Session token usage is skewed: in both traces, the top 25 % of sessions contribute more than 80 % of the tokens (Figure 8).

As shown in Figure 8, the computation required varies significantly across sessions, with a small fraction of sessions accounting for most of the tokens to compute. Nevertheless, thanks to the large request volume, the skewed computation can still be balanced across all instances:

Finding 6: Despite the skewness, the token load can still be balanced across serving instances over time.

To quantify this, we use a trace-driven simulation that replays all requests of a trace in arrival order on a simulated cluster with the same number of instances as the traced cluster. The simulator dispatches each request to the instance

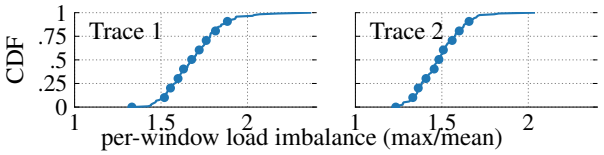


Figure 9: Per-window token load imbalance (max/mean) across instances under load-balancing dispatch, for two traces.

```
req = receive()
c = kvc.hit_len(req)    # local KV$ hit, per instance
l = load()              # load, per instance

sched_to = instances.argmin/max(f(c, l))
req.forward(sched_to)
```

Figure 10: A high-level overview of the existing LLM scheduling scoring methods. $f(c_i, l_i)$ scores instance i by the request's KV$ reuse on it (c_i) and its load (l_i).

currently holding the fewest in-flight tokens, counting all tokens—prefill and decode—of the requests running on that instance. Each request stays at its instance for the execution time recorded in the trace. Figure 9 shows the resulting load balancing profile: for each one-minute window, we report the token count of the most loaded instance divided by the mean token count across all instances (max/mean). The load stays largely balanced: the median per-window imbalance is only 1.7 for Trace 1 and 1.5 for Trace 2, and even the worst window stays below 2.4 and 2.1, respectively. The simulation is approximate—a request's execution time on the simulated cluster may differ from that on the original cluster—but it suffices to reveal the workload's balance potential, and our analysis in §4.1 further confirms that schedulers balancing the load alone keep instance loads balanced.

Balance holds because concurrent requests far outnumber instances: the number of requests arriving within a 1-minute window is on average 43× (Trace 1) and 48× (Trace 2) the number of serving instances, and never drops below 30× in either trace. Hence, even when token-heavy requests occupy a subset of instances, the remaining instances absorb many token-light ones, which evens out the skew of individual sessions. Such a high request-to-instance ratio is common in serving: a GPU instance requires a large batch of requests to achieve high utilization, and requests are not scarce for large service providers like BAILIAN.

4 Balanced session-centric scheduling

4.1 Analysis of existing schedulers

A preliminary on LLM scheduling in a cluster. To the best of our knowledge, existing LLM schedulers [75, 53, 37, 2, 45, 3, 59, 82, 74, 11, 65] route requests by *per-request scoring*, as Figure 10 illustrates: upon receiving a request, the scheduler computes a score for each instance and routes the request to the instance with the lowest (or highest, depending on the definition) score. The score of

instance i is typically a function $f(c_i, l_i)$ of two LLM-specific indicators: l_i describes the instance's current state, typically its load, e.g., its batch size or the number of inflight tokens. c_i estimates the computation the request would need on the instance, typically by the prefix hit length of the request's prompt in instance i's local KV$. Methods mainly differ in how they define f, i.e., how they weigh the instance's load against the request's KV$ reuse. Our motivating analysis uses the following representative methods, while §5.1 covers more:

- BAILIAN is the production scheduler deployed at BAILIAN. It scores an instance by a weighted sum of its load and the request's KV$ miss: $f = (1-\lambda)\cdot\hat{l}_i + \lambda\cdot(1-\hat{c}_i)$ (the lower the better), where $\hat{l}_i$ is the instance's batch size and $\hat{c}_i$ is the request's KV$ hit ratio on the instance. Each indicator carries a $\hat{\cdot}$ because it is normalized to a common range; otherwise the two cannot be added. The weight λ is tuned to trade KV$ reuse for load balance; for agentic workloads, the operator sets a large λ to favor KV$ reuse. Popular open-source schedulers adopt a similar design [45, 3].

- *LMetric* [75] is a recent scheduler that multiplies the two indicators instead of adding them for scoring instances: $f = (q_i + L - c_i) \times l_i$, where L is the request's prompt length and q_i is the prefill work already queued at instance i, so $q_i + L - c_i$ is the prefill computation left before the request's first token.

- *Load-balance-only* considers load only, i.e., $f = l_i$: it routes each request to the least-loaded instance regardless of its KV$ reuse. It is a variant of the classic join-the-shortest-queue policy used in traditional cluster-level request routing, and the default policy of vLLM [61].

The analysis methodology. To understand how representative methods perform when serving agents, this section compares the methods by replaying Trace 1 on a 32-instance vLLM [61] cluster serving Qwen3-Coder-30B-A3B under PD colocation, with Mooncake [53] and LMCache [38] as the global KV$ store; §5.1 details the testbed, its tuning, and how the replay keeps the KV$ reuse consistent with the trace. We replay only Trace 1, since the two traces exhibit similar patterns (§3); PD disaggregation shows similar results (§5.3).

We report each method's *TPS within SLO*: the cluster-wide output TPS (output tokens generated per second) contributed only by requests that satisfy both the TTFT and TPOT SLOs. The request-level SLO matters for agents: it determines the task completion time, and a response that is too slow makes the agent treat the request as timed out [8]. The SLO follows common practice [80, 64, 76]: a TTFT SLO that grows linearly with the prompt length, $b + a \cdot L$ for a prompt of L tokens, that is, a fixed budget b plus a per-token budget a; and a fixed TPOT SLO.

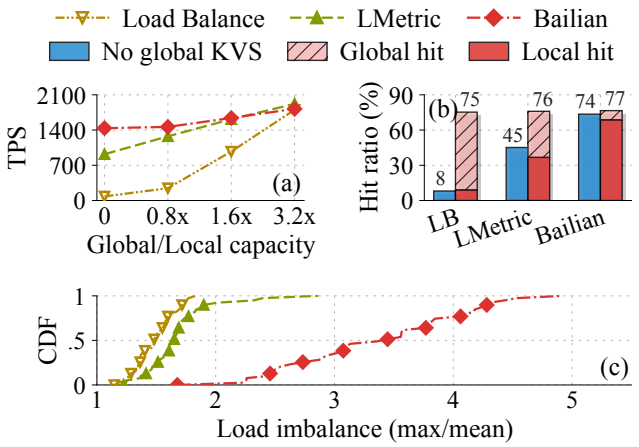


Figure 11: (a) TPS within SLO under different global-tier offerings. (b) KV$ reuse ratio and its source. (c) Distribution of the per-30-second-window load imbalance.

We compare the methods at one offered request rate per global-tier provisioning, the same rate for every method, under a fixed SLO; §5 varies the rate and the SLO, with consistent results. To further measure the methods under different KV$ pressure, we vary the provisioning of the global tier. We call the global tier *fully provisioned* when its capacity is large enough that a request routed to any instance can usually fetch its session's KV$ from it. Our testbed's global tier can cache nearly all ongoing sessions, so a fully provisioned tier always serves the KV$; in production, however, we do observe under-provisioned global tiers, with insufficient capacity or bandwidth.

The KV$-awareness vs. load balance trade-off. Existing schedulers still trade KV$ reuse for load balance. LMetric [75] reports KV$ reuse ratios comparable to those of other KV$-aware schedulers while improving load balance, on workloads with ideal KV$ reuse ratios typically around 50–70 %. Our agentic serving workload, with an ideal reuse ratio of 82 % (Finding 1), shows that these two benefits do not always hold together: without a global tier, LMetric's reuse ratio is only 45 %, compared with BAILIAN's 74 % (Figure 11 (b)). As a result, without a global tier, LMetric serves only 920 TPS within SLO, 36 % below BAILIAN's 1,437 TPS (Figure 11 (a)), even though LMetric achieves better load balance than BAILIAN's linear combination (Figure 11 (c)). We measure each instance's load as its average number of running requests within a 30 s window. Load imbalance is the ratio of the maximum load to the mean across the 32 instances [51, 43].

Although the ideal reuse ratio of the agentic workload is only about 10 percentage points above the high end of the non-agentic ones (82 % vs. 70 %), the difference in prefill computation is far larger: raising the reuse ratio from 70 % to 80 % cuts the prompt tokens that must still be computed by one third.

Load balance does not always trade away KV$ reuse. We do observe opportunities for improvement over policies that favor KV$ reuse: with a fully provisioned global tier, load balancing need not reduce KV$ reuse. This is because the global tier can serve the KV$ missed in the local tier, so even a pure load-balance method achieves a high reuse ratio (Figure 11 (b)). As shown in Figure 11 (a), the TPS within SLO of a load-balance-only solution is within 7 % of the best of the three methods.

Local hits are still important. Although KV$ hits from the global tier can recover reuse, the local reuse is still important: Load-balance-only and BAILIAN both reuse about 76 % of prefill tokens, yet their TPS is similar with sufficient global-tier capacity (Figure 11 (a, b)), despite that Load-balance-only has a better load balance: 1.5× vs. 3.3× imbalance as shown in Figure 11 (c). This is because the local hit ratio of Load-balance-only is only 9.1 %, vs. BAILIAN's 68.8 %, requiring more global-tier transfers that consume bandwidth and incur latency.

Takeaway: Scheduling should maintain load balance and KV$ reuse while preserving as much local reuse as possible.

4.2 Session-centric scheduling: challenges and our approach

Opportunity: session-centric scheduling can reconcile load balance and KV$-awareness. Our insight is that routing by session, rather than by request, can achieve both objectives. Suppose the scheduler balances sessions across the cluster and routes each follow-up request to the instance serving its session. This strategy can balance the load without losing reuse, because the token load can still be balanced at scale despite the skew of sessions, as characterized in §3.2 (Finding 6). Note that placing sessions is coarser than placing requests, which requires further care to balance the load (as we describe in *Challenge# 2* below in more detail). It also keeps (local) KV$ reuse high: every request except the first reuses the KV$ of earlier requests in the same session (Finding 2), and the reuse comes quickly (Finding 4), so the KV$ usually still resides in the local tier. The first-turn request is the only exception, and it loses little performance because its reusable KV$ is mostly the shared system prompt (Finding 3), which the global tier serves. The load on the global tier also stays low, because in the common case only first-turn requests read from it.

Realizing session-centric routing faces two challenges:

#Challenge 1: missing session information. Existing session-centric schedulers like llm-d [37] require each request to carry a session identifier (ID), which is impractical when a provider serves agents, for two reasons. First, requests carry no session identifier. The APIs that agents use, e.g., Anthropic's Messages API and OpenAI's Chat Completions API [7, 46], are stateless: each request resends the session's full history and carries no identifier; stateful variants [47] are optional and disabled under zero-data-retention policies. Nor can the provider add one: agents send requests through diverse frameworks (e.g., Claude Code [6] and OpenClaw [48]), and it is impractical for these frameworks to agree on a session protocol. Second, the router cannot infer the session from the request's source: at BAILIAN, enterprise customers send requests through global proxies, which anonymize the source (e.g., the IP).

One possible solution is to let the router infer sessions by itself, e.g., by recording past requests and matching each new request against the records. This is impractical: existing routers keep only per-instance state, i.e., the indicators in Figure 10. Per-session state, in contrast, scales with the number of active sessions, which far exceeds the number of instances the router manages, and must be updated as sessions come and go. This complicates the router, and to our knowledge no existing LLM scheduler adopts it.

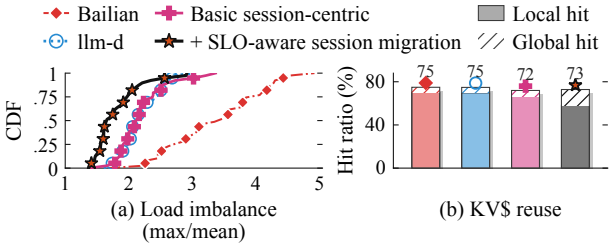


Figure 12: (a) Per-window load imbalance, as in Figure 11 (c). (b) KV$ reuse ratio and its source. Each arm runs at the request rate where its group peaks in TPS within SLO.

#Challenge 2: unconditional sticking still imbalances the load. Sticking every request to its session's instance can still imbalance the load, because sessions differ widely in size (§3.2) and the size is unknown when the session is first routed. Suppose several sessions placed on the same instance all turn out to be long, with many turns: each follow-up turn returns to the instance with a longer prompt to prefill, so the instance overloads as the sessions grow. Figure 12 measures this on our testbed. llm-d [37] routes requests by session ID, which we reconstruct offline from the trace and supply with each request (§3). Its mean imbalance is still 2.17×, though it is more balanced than BAILIAN's linear combination of KV$-awareness and load (3.34×).

Basic solution: the turn and the KV$ hit identify the session without per-session state. To address the first challenge, we observe that session-centric scheduling leverages session information to answer two questions per request for routing: (1) whether the request starts a session and thus should be load-balanced, and (2) for a follow-up request, which instance served the session's previous request, so that the request can stick to it for KV$-awareness. Both can be answered without per-session state. Specifically, the first can

```
req = receive()                          (a) stateless session-centric routing
c = kvc.hit_len(req)   # local KV$ hit, per instance
q = queued_cost()  # sum of prefill_cost in i's queue
prev = instances.argmax(c)  # likely served last turn

if req.turn != 0 and session_cached(req, prev) \
        and (meets_ttft(req, prev)          # (b)
             or none_meets_ttft(req)):
    sched_to = prev                         # stick
else:                                       # balance
    sched_to = instances.rr_argmin(
        key=lambda i: estimate_load(req, i))
req.forward(sched_to)

def session_cached(req, i): # i caches req's session
    return c[i] > HIT_RATIO * est_hit(req)
    # est_hit: the hit expected from req's history
```

```
def meets_ttft(req, i):                  (b) SLO checks
    # the deadline check
    ttft = estimate_load(req, i) / PREFILL_RATE
    return ttft <= SLACK * ttft_slo(req)

def none_meets_ttft(req):  # the last resort:
    # no instance meets the deadline, so stick
    return not any(meets_ttft(req, i)
                   for i in instances)
```

```
def estimate_load(req, i):               (c) load model
    # i's queued prefill + req's own
    return q[i] + prefill_cost(req, c[i])

def prefill_cost(req, hit): # Eq. 1, reusing hit
    L = req.prompt_len; n = L - hit  # new tokens
    return C_LIN * n + C_ATT * n * (L - n / 2)
```

Figure 13: Simplified pseudocode of SMETRIC. `C_LIN` and `C_ATT` are constants fitted by profiling the model. `PREFILL_RATE` (the instance prefill rate) is measured, while `SLACK` and `HIT_RATIO` are the two hyperparameters.

be answered via the request's turn: a first-turn request marks the arrival of a new session. Since requests carry the session's full history (§2), the router deduces the turn from the number of historical messages the request carries. The deduction fails only if the agent drops part of its history, e.g., by compaction. The failure is benign: such a request hits little KV$ anyway, so routing it by load balance suffices. The second can be answered via the local KV$ hit: a follow-up request mostly reuses the KV$ of its own session (Finding 2), and the reuse comes quickly (Finding 4), so the instance with the highest local hit is likely the one that served the previous request.

Figure 13 (a) shows our basic policy. It assumes PD disaggregation and routes prefill requests only; the extension to PD colocation is similar (though a little more complex) and is left to §B due to space limitations. The router first checks whether a request is a first-turn request (line 5). If so, it routes the request to the least-loaded instance, which balances sessions across the cluster (lines 9–11); otherwise it sends the request to the instance with the highest KV$ hit (lines 4 and 8)—the instance that likely executed the session's previous request. The load in the balance branch is estimated by the model in (c), which subtracts the request's local hit from its prefill work, so the routing decision stays KV$-aware: a first-turn request still favors the instances that cache its system prompt. As shown in Figure 12, the basic policy achieves a load-imbalance curve and KV$ reuse similar to session-centric scheduling (llm-d) with session IDs.

Two cases are worth mentioning. First, a follow-up request whose session has been evicted from the local tier is treated as a first-turn request, since there is no local KV$ it can use. `session_cached` detects eviction by comparing the highest local hit against the hit expected from the request's history (`est_hit`, which excludes the newly appended turn that no instance has cached); a large gap indicates that the local tier has evicted the session. `HIT_RATIO` is a pre-configured threshold below one that tolerates small differences between the estimate and the real hit; SMETRIC is robust to it (§5.5). Second, SMETRIC sticks a follow-up request to its session's instance only if that instance can finish it within the SLO, which we elaborate next.

Extending the basic policy: SLO-aware session migration via scheduling. To address the second challenge, SMETRIC sticks a follow-up request to the instance that served its previous request only if that instance can finish the request within its SLO. Otherwise, SMETRIC routes the request to the least-loaded instance, which migrates the session away to prevent SLO violations due to overloading. Figure 12 shows the effect: under the same run, migration lowers the basic policy's mean imbalance from 2.18× to 1.81× and leaves no window above 3×. The migrated request still reuses its KV$: the target instance fetches the session's KV$ from the global tier over RDMA (③ in Figure 2). Note that under high load (or bursts), no instance may be able to meet the deadline. SMETRIC then still routes the request by KV$-awareness to add minimal new load to the cluster (line 7).

`meets_ttft()` in Figure 13 (b) shows the detailed implementation of the check. We use the request's SLO attainment, rather than the instance's load, as the indicator for migration. A load-based indicator would also work here, e.g., when the instance's load exceeds the others' by a margin, we migrate the session off that instance by routing the request away from it. However, the load threshold is difficult to set: too strict a threshold migrates sessions that the instance could still serve on time and loses their local KV$ reuse, while too loose a threshold rarely triggers migration. The SLO sets the threshold directly: a request migrates exactly when sticking would miss its deadline. The TTFT deadline (`ttft_slo`) is a function set by the provider, typically linear in the request length [80, 64, 76], and `SLACK` is a pre-configured hyperparameter that scales the deadline to leave headroom, to which SMETRIC is also robust over a common range (§5.5). The check compares the deadline against the estimated prefill time of the instance's queued requests plus the new request, which we describe next.

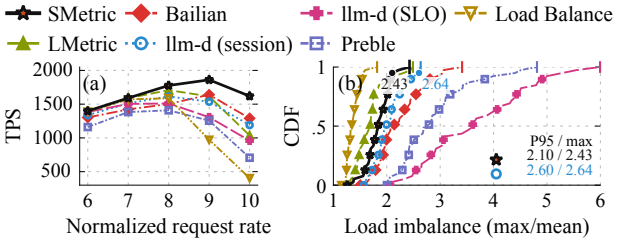


Figure 14: (a) TPS within SLO and (b) distribution of the per-window load imbalance (§4.1) on the 32-instance Qwen3-30B-A3B PD-colocated setting.

Estimating the load. Both load balancing and the SLO check need a load model that is fast to compute yet reasonably accurate. Existing schedulers like Dynamo [45] use the number of tokens to prefill as the load, which we found inaccurate because the computation of each token depends on its context [81]. We therefore estimate the load (Figure 13 (c)) in a more precise form, following prior work [12]:

$$\text{cost}(n, L) = c_{\text{linear}}\, n + c_{\text{attn}}\, n\,(L - n/2), \qquad (1)$$

where L is the request's context (prompt) length and n is the number of new tokens to prefill. c_{linear} and c_{attn} are coefficients fitted by profiling, and the term $n(L - n/2)$ follows from causal (lower-triangular) attention: each new token attends to all tokens before it, on average $L - n/2$ of them. §5.4 quantifies the contribution of this model.

5 Performance evaluation

5.1 Evaluation setup

Testbed. Our testbed is a cluster of four GPU servers, each equipped with eight NVIDIA H20 GPUs (96 GB HBM), 160 CPU cores, and 1280 GB of host DRAM. Servers are interconnected with 200 Gbps RDMA NICs. Each instance runs the latest vLLM [28, 61]. The global KV$ store is based on Mooncake [53] and LMCache [38], the state-of-the-art global-tier KV$ store described in Figure 2. We tuned each system to its peak performance: e.g., KV$ fetches from the global tier nearly saturate the RDMA bandwidth. Unless otherwise mentioned, we use all instances to evaluate the performance, and fully provision the global tier; §5.2 and §5.3 also evaluate with different global KV$ capacities.

Evaluated model. We evaluate two models, Qwen3-Coder-30B-A3B and Qwen3-235B-A22B-FP8, both popular models on the market [67]. Qwen3-30B can be served with only one GPU, allowing us to evaluate up to 32 instances. In contrast, Qwen3-235B requires at least 4 GPUs per instance, so we can evaluate up to 8 instances.

Evaluated workload. We replay Trace 1 (§3) for all experiments. Because the requests are collected from a much larger cluster than our testbed, we sample the requests for the evaluations following prior work [75, 80, 41, 21, 56], keeping

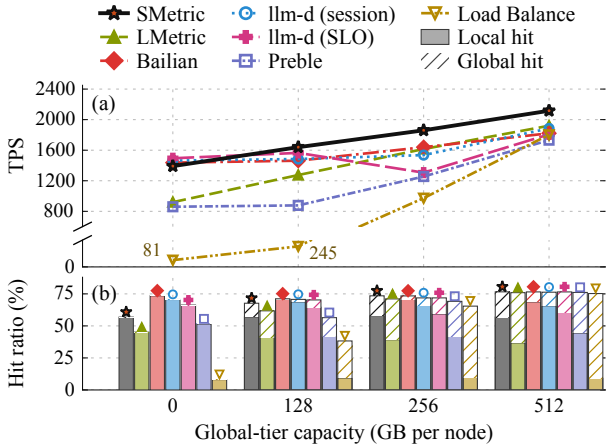


Figure 15: (a) TPS within SLO and (b) KV$ reuse ratio and its source under different global-tier capacities on the 32-instance Qwen3-30B-A3B PD-colocated setting.

every request of a session sampled so that its KV$ reuse chain stays complete. We report the request rate as a multiple of a base rate, a small fixed fraction of the trace that we cannot disclose due to privacy requirement of BAILIAN. One issue with replaying is that the evaluated model's responses differ from those recorded in the trace. Such a difference breaks the KV$ reuse pattern in two ways: the generated response may have a different length, and a follow-up request's recorded history no longer matches the tokens the serving instance actually cached. We fix the issue as follows. First, we force the evaluated model to generate exactly as many tokens as the recorded response, by ignoring the end-of-sequence token and capping the generation at the recorded length. Second, we rewrite part of the follow-up requests with the generated content to ensure consistent KV$ reuse.

5.2 End-to-end performance: PD-colocation

Compared targets. We compare SMETRIC with the state-of-the-art schedulers described in §4.1, as well as other schedulers from well-known open-source projects and research work. llm-d (session) [37] routes requests by session ID; it is the session-centric baseline described in §4.2. We supply the session ID reconstructed offline. llm-d (SLO) routes by predicted latency: an online-trained model predicts the TTFT and TPOT the request would see on each instance, and the request goes to the instance with the least predicted headroom among those meeting the SLOs we score against. Other non-open-source schedulers like the Mooncake scheduler [53] follow a similar rationale. Preble [59] filters instances by KV$ hit before scoring them: a request whose cached prefix covers more than half of its prompt goes to the least loaded instance holding that prefix, and any other request to the instance with the least estimated work, a linear-combined (BAILIAN-like) score. We use its implementation in AIBrix [2].

TPS and per-token latency. Figure 14 (a) reports the TPS

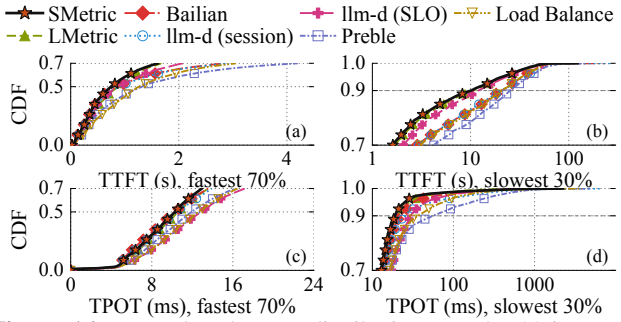


Figure 16: Per-token latency distributions on the 32-instance Qwen3-30B-A3B PD-colocated setting. (b, d) use a log scale.

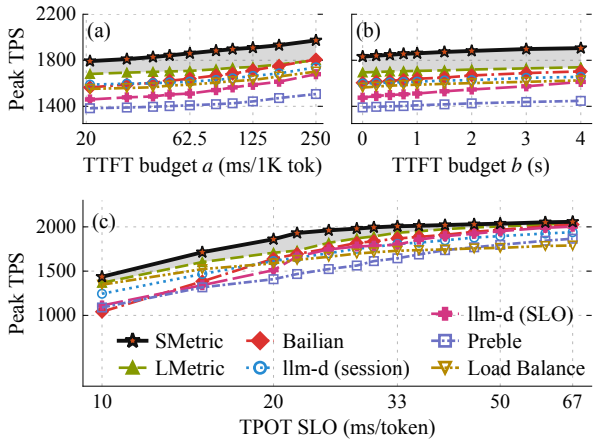


Figure 17: Sensitivity to SLO configurations on the 32-instance Qwen3-30B-A3B PD-colocated setting.

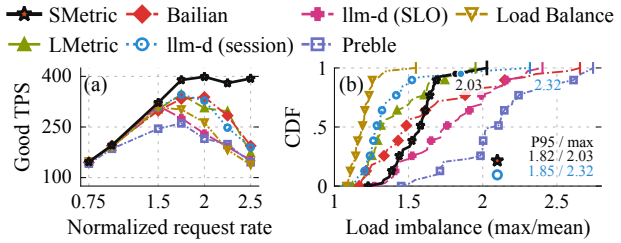


Figure 18: (a) TPS within SLO and (b) distribution of the per-window load imbalance (§4.1) on the 8-instance Qwen3-235B-A22B-FP8 PD-colocated setting.

within SLO (§4.1) on Qwen3-30B with all 32 instances under the fully provisioned store, at increasing request rates and a fixed SLO (TTFT budgets $b = 1$ s and $a = 62.5$ ms per 1K prompt tokens; TPOT within 20 ms). SMETRIC achieves the highest TPS within SLO: its peak exceeds the best baseline's by 9 % (1860 vs. LMetric's 1708 tok/s, reached one rate lower). The improvement over the best baseline grows with load: within 1 % at 6× load, +4 % at 8×, +13 % at 9×, and 26 % past saturation (10×), where every scheduler loses TPS and routing mistakes turn directly into SLO violations. The result is consistent across global-tier provisionings (see Figure 15 (a)), except for setups without any global KV$ store. This is acceptable, because deploying the global store for serving agents is now common practice. The per-token

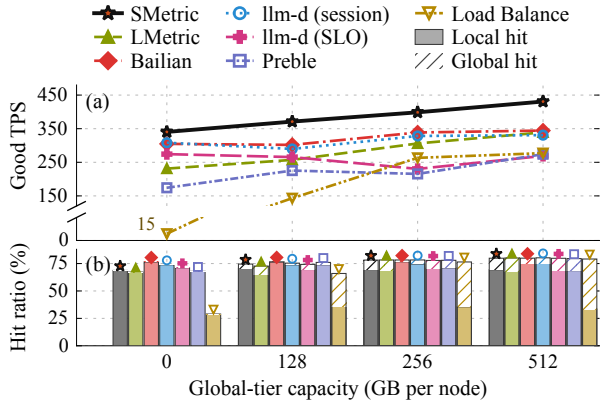


Figure 19: (a) TPS within SLO and (b) KV$ reuse ratio and its source under different global-tier capacities on the 8-instance Qwen3-235B-A22B-FP8 PD-colocated setting.

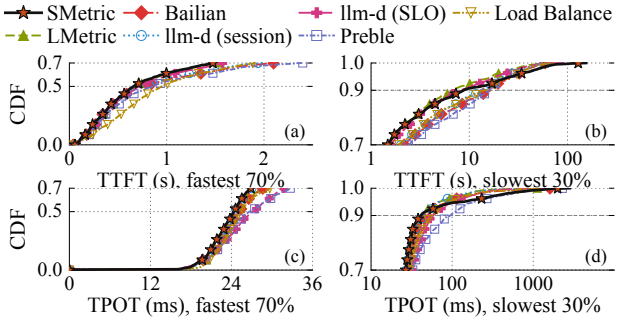


Figure 20: Per-token latency distributions on the 8-instance Qwen3-235B-A22B-FP8 PD-colocated setting. (b, d) use a log scale.

latency in Figure 16 shows similar results: SMETRIC has the lowest TTFT at every reported percentile and the lowest P90 TPOT.

SMETRIC serves better because it keeps local KV$ hits and load balance simultaneously, without trading away the overall reuse ratio. Figure 15 (b) shows that SMETRIC matches the highest reuse ratio (BAILIAN's, 73 % for both) and serves most of it from the local tier (58 %). Figure 14 (b) shows that it does so without trading load balance.

Sensitivity to the SLO. Figure 17 further compares the TPS within SLO under various SLO setups, as different applications may have different SLO requirements. SMETRIC is robust across all SLO setups and achieves the best TPS among all baselines: it is 7–10 % above the best baseline across different TTFT budgets, and 5–12 % at TPOT SLOs of 10–22 ms. As the TPOT SLO becomes looser, the gap between SMETRIC and others narrows, because a loose SLO can tolerate more computation or queuing, but SMETRIC can still achieve a lower latency thanks to better scheduling.

Performance on a larger model. To see how SMETRIC performs under larger models where each instance requires multiple GPUs, we repeat the evaluation of the 30B model with Qwen3-235B on eight instances, with the TPOT SLO

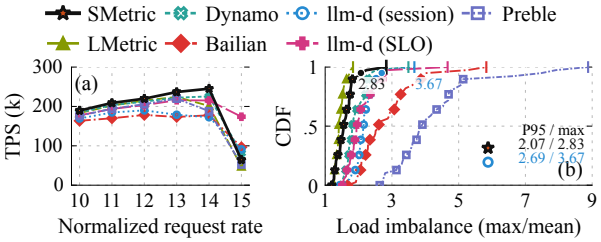


Figure 21: (a) Prefill TPS within SLO and (b) distribution of the per-window load imbalance (§4.1) on the 32-instance Qwen3-30B-A3B PD-disaggregated setting.

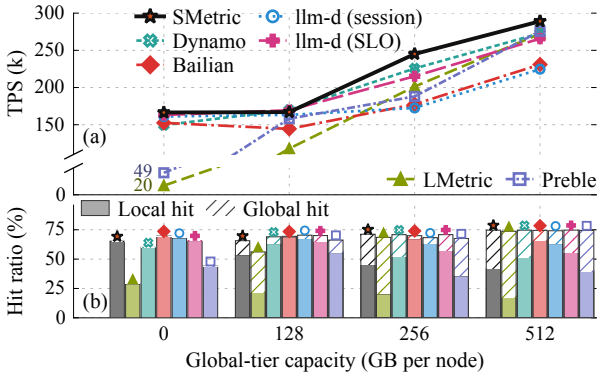


Figure 22: (a) Prefill TPS within SLO and (b) KV$ reuse ratio and its source under different global-tier capacities on the 32-instance Qwen3-30B-A3B PD-disaggregated setting.

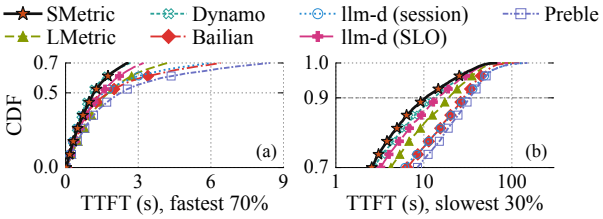


Figure 23: TTFT distributions on the 32-instance Qwen3-30B-A3B PD-disaggregated setting for (a) common and (b) tail requests. (b) uses a log scale.

relaxed to 33 ms, as the larger model decodes slower. As shown in Figure 18 (a), the results are consistent with those on Qwen3-30B: SMETRIC again achieves the highest TPS within SLO, with a peak 15 % above the best baseline's (399 vs. LMetric's 346 tok/s), a gap that widens to 28 % as the load grows (2.25×), the lowest median TTFT and P90 TPOT (Figure 20), and consistent results across different SLO configurations (Figure 28 in the appendix). Serving the large model is similar to serving smaller models from the perspective of scheduling, because the difference is at the instance level, whereas scheduling routes requests across instances. SMETRIC still keeps the KV$ reuse and the load balance as before: it matches the highest reuse ratio (78 %, the same as LMetric's) and serves 70 % of the prompt tokens from the local tier (Figure 19 (b)), and its load imbalance stays close to LMetric's (Figure 18 (b)); only Load-balance-only, which optimizes that quantity directly, is more balanced, and it serves 34 % less TPS at this rate (23 % peak to peak) because it reuses KV$ mostly from the global tier (Figure 19 (b)).

Nevertheless, we do make two observations on large models. First, the largest performance gap widens (15 % vs. 9 % on Qwen3-30B) because the larger model puts more pressure on the global tier, which makes it more important for the scheduler's decision to favor local reuse. Second, llm-d (session) appears more balanced than SMETRIC in Figure 18 (b), as its curve stays left of ours up to the P90 window. Despite this, SMETRIC can still achieve a better TPS because, among the KV$-aware schedulers, it has a lower tail imbalance: the P99 max/mean imbalance of llm-d (session) reaches 2.20× and its worst 2.32×, against 1.99× (P99) and 2.03× for SMETRIC, respectively. This tail imbalance is critical to the SLO attainment, because the requests queued at an overloaded instance are the ones that miss the SLO. SMETRIC migrates the session away from such an instance (§4.2), which reduces the latency increase under extreme imbalance.

5.3 End-to-end performance: PD-disaggregation

Evaluation setup and compared targets. We next evaluate SMETRIC under PD disaggregation. Since KV$ reuse only affects prefill, we deploy 32 prefill instances of Qwen3-30B, run only the prefill phase, and report the *prefill TPS* within SLO—the number of prompt tokens whose KV$ is produced per second, computed or reused from a hit—and the latency results (TTFT). We use only the single-card model (30 B) to evaluate at a larger scale; the results on the multi-card model (235 B) are similar.

The compared targets are the same as under PD colocation, except that we further evaluate Dynamo [45], a state-of-the-art system, which optimizes upon Load-balance-only: it routes a request to the instance with the fewest tokens left to prefill, including the queued tokens of other requests and the new tokens of the request to route considering KV$ hits.

Prefill TPS and TTFT. Figure 22 (a) compares the prefill TPS across global-tier provisionings, and Figure 21 (a) evaluates how the TPS changes with the request rate under the fully provisioned store. SMETRIC still achieves the highest prefill TPS at three of the four provisionings, 2–9 % above the best-performing baseline; only with the 128 GB store does it slightly underperform Dynamo and llm-d (SLO), by 2 %. The peak TPS achieved by SMETRIC across request rates exceeds Dynamo's peak by 9 % (245 vs. 226 ktok/s, Figure 21 (a)). The latency results (TTFT) in Figure 23 show similar results: SMETRIC delivers the lowest tail TTFT, with a P90 19 % lower than Dynamo's, the closest baseline, and with a close median.

We found that balanced schedulers matter more for TPS under PD disaggregation: Dynamo and LMetric now outperform or are similar to the KV$-aware scheduler (BAILIAN).

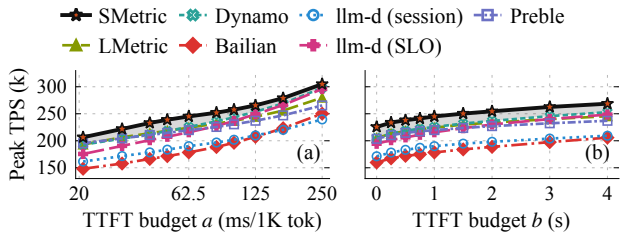


Figure 24: Sensitivity to SLO configurations on the 32-instance Qwen3-30B-A3B PD-disaggregated setting.

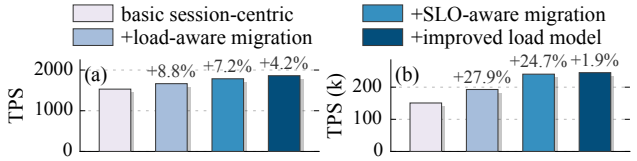


Figure 25: Ablation study of SMETRIC on (a) the PD-colocated and (b) the prefill instances of the PD-disaggregated 32-instance Qwen3-30B-A3B setups.

The reason is that prefill is more compute-intensive, so an imbalanced load directly affects the prefill TPS, whereas decode can run the queued requests in a larger batch to amortize the imbalance cost under PD colocation. Nevertheless, achieving similar KV$ reuse with load balancing, as SMETRIC does, is still beneficial: with the 256 GB store it matches the best baselines' reuse ratio (71 %, Figure 22 (b)) with a load nearly as balanced as LMetric's: averaged over the 30 s windows, the busiest prefill instance holds 1.6× as many running and waiting requests as the average instance, against 1.9× under Dynamo and 2.8× under BAILIAN (Figure 21 (b)).

Sensitivity to the SLO. Figure 24 reports the prefill TPS within SLO under various SLO setups. The results are similar to those under PD colocation: SMETRIC still achieves the best TPS: 6–10 % above the best baseline across different fixed TTFT setups and 2–9 % across different per-token budgets.

We also repeat this evaluation on an open agentic-coding trace, the Codex agent traces on SWE-Bench Pro [24], and reach the same conclusions: SMETRIC achieves the highest peak prefill TPS, 10 % above the best baseline, and the lowest TTFT (§D).

5.4 Ablation study

This section conducts an ablation study on the routing designs of SMETRIC. We use the 32-instance Qwen3-30B-A3B setups above, under PD colocation and PD disaggregation, each at the request rate where SMETRIC achieves peak TPS.

As shown in Figure 25, our starting point is *basic session-centric*, which only uses the request's turn and its local KV$ hit (lines 2, 4 and 5 of Figure 13) to distinguish sessions and realize basic session-centric scheduling (similar to llm-d (session)). First, adding overloading migration, i.e., replacing the SLO-aware migration on lines 6–7 with a load-aware filter—a follow-up request leaves its session's instance when that instance's load (queued prefill plus ongoing decode) exceeds the cluster mean by a margin—improves the TPS by 8.8 % and 27.9 % under colocation and disaggregation, respectively, confirming that session-centric scheduling alone cannot handle overloading cases. Second, replacing the load-aware filter with our SLO-aware migration (lines 16–23 in Figure 13) further improves the TPS by 7.2 % and 24.7 %, showing that the SLO is a more accurate migration signal for TPS. Finally, replacing the load estimate in the SLO calculation from token counts to our cost model described in §4.2 (Eq. 1) increases the TPS by 4.2 % and 1.9 %, showing that a more accurate performance model improves scheduling. Note that using a performance model alone (e.g., llm-d (SLO)) is still insufficient, as we have evaluated in §5.2 and §5.3.

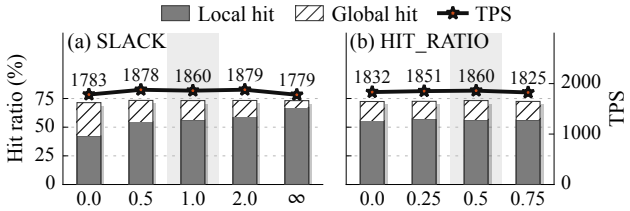


Figure 26: Sensitivity to the two hyperparameters of SMETRIC on the 32-instance Qwen3-30B-A3B PD-colocated setup.

5.5 Sensitivity to the two routing hyperparameters

SMETRIC uses two hyperparameters in Figure 13: `SLACK` controls to what extent it can tolerate SLO violations and `HIT_RATIO` determines when the local cache is treated as evicted for the request. Figure 26 conducts a sensitivity analysis on these parameters. SMETRIC is robust to both as long as they are set to reasonable values: TPS within SLO moves by 1.0 % over `SLACK` from 0.5 to 2 and by 1.9 % over the whole range of `HIT_RATIO`, except for the improper values, e.g., setting `SLACK` to 0 disables session stickiness (every follow-up request migrates), while setting it to ∞ completely ignores the SLO checks.

6 Related Work

LLM request scheduling. Recent work schedules LLM requests with KV$-aware, SLO-aware, load-balancing, fairness, and output-length-aware policies [59, 53, 60, 82, 11, 55, 65, 23, 58, 57]. SMETRIC builds on these policies and derives agent-session hints for the scheduler without requiring LLM requests to provide session information. Unlike systems that co-design serving with agent applications and require applications to expose inter-request structure [33, 39, 73, 13], SMETRIC operates entirely on the provider side and treats agents as unmodified clients, matching a typical serving scenario today. AgentKV [77] also targets agentic serving but learns agent execution to decide which KV$ to keep; SMETRIC needs no learning and infers the session from two indicators the router already holds.

KV$ reuse and optimizations. Reusing KV$ across requests is well studied, and both industry and academia have

built mature reuse mechanisms [18, 19, 28, 69, 70, 79, 72, 49], We build on these mechanisms and focus on scheduling to better utilize them. Other work further reduces the memory footprint and transfer cost of KV$ with system–algorithm co-designs: CacheGen compresses and streams KV states [35, 29, 20, 17, 66], and quantization and compression methods reduce KV$ memory pressure [36, 22, 30, 10, 78]. These techniques are orthogonal to SMETRIC.

Optimizing LLM serving. Scheduling is one part of the serving stack, and much work targets its other layers, including efficient attention algorithms [14, 15], optimized attention kernels and KV layouts [71, 52, 25], elastic resource scaling [67, 76], and many others [26, 32, 28, 62, 40, 31, 27]. They are orthogonal to SMETRIC, which is at an upper layer, and we believe that as long as LLMs still reuse KV$, our techniques and findings are effective.

7 Conclusion

Agentic serving demands high cluster-wide TPS and makes KV$ reuse dominant, requiring schedulers to be both load-balanced and KV$-aware. SMETRIC achieves the best of both worlds by first routing requests by session, with session information deduced statelessly at the router, and further deriving an SLO-aware migration scheme to prevent imbalance caused by unpredictable session growth. It improves the peak cluster TPS by 9–15 % under prefill-decode colocation and the peak prefill TPS by 9 % under disaggregation over state-of-the-art schedulers, at lower latency.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA, July 2024. USENIX Association.

[2] Aibrix. https://github.com/vllm-project/aibrix, 2025.

[3] Aigw. https://github.com/aigw-project/aigw, 2025.

[4] Anthropic. Claude code: Sub-agents and agent teams. https://docs.anthropic.com/en/docs/claude-code/sub-agents, 2024.

[5] Anthropic. Building a C compiler with Claude. https://www.anthropic.com/engineering/building-c-compiler, 2025.

[6] Anthropic. Claude code. https://code.claude.com/docs/en/overview, 2026.

[7] Anthropic. Claude platform docs: Using the Messages API. https://platform.claude.com/docs/en/build-with-claude/working-with-messages, 2026.

[8] Anthropic. Elevated error rates on Opus 4.6. Anthropic Status Page, 2026.

[9] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do ai agents spend your money? analyzing and predicting token consumption in agentic coding tasks, 2026.

[10] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic KV cache compression based on pyramidal information funneling. *CoRR*, abs/2406.02069, 2024.

[11] Siyuan Chen, Zhipeng Jia, Samira Khan, Arvind Krishnamurthy, and Phillip B. Gibbons. Slos-serve: Optimized serving of multi-slo llms. *CoRR*, abs/2504.08784, 2025.

[12] Rongxin Cheng, Yuxin Lai, Xingda Wei, Rong Chen, and Haibo Chen. KUNSERVE: parameter-centric memory management for efficient memory overloading handling in LLM serving. In Antonio Barbalace, Luo Mai, Roxana Geambasu, and Peter R. Pietzuch, editors, *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, pages 1244–1260. ACM, 2026.

[13] Yinwei Dai, Zhuofu Chen, Anand Iyer, and Ravi Netravali. Aragog: Just-in-time model routing for scalable serving of agentic workflows. *CoRR*, abs/2511.20975, 2025.

[14] DeepSeek-AI. Deepseek-v3 technical report. *CoRR*, abs/2412.19437, 2024.

[15] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. Technical report, 2026. Available at Hugging Face model repository. Accessed: 2026-06-10.

[16] Haoran Ding, Zhaoguo Wang, and Haibo Chen. Fm-agent: Scaling formal methods to large systems via llm-based hoare-style reasoning, 2026.

[17] Shaoting Feng, Hanchen Li, Kuntai Du, Zhuohan Gu, Yuhan Liu, Jiayi Yao, Siddhant Ray, Samuel Shen, Yihua Cheng, Ganesh Ananthanarayanan, and Junchen Jiang. Adaptcache: KV cache native storage hierarchy for low-delay and high-quality language model serving. *CoRR*, abs/2509.00105, 2025.

[18] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with cachedattention. In Saurabh Bagchi and Yiying Zhang, editors, *Proceedings of the 2024 USENIX Annual Technical Conference, USENIX ATC 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 111–126. USENIX Association, 2024.

[19] In Gim, Guojun Chen, Seung-Seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In Phillip B. Gibbons, Gennady Pekhimenko, and Christopher De Sa, editors, *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024.

[20] Nikhil Gopal and Kostis Kaffes. Harvest: Opportunistic peer-to-peer GPU caching for LLM inference. *CoRR*, abs/2602.00328, 2026.

[21] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving dnns like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 443–462. USENIX Association, 2020.

[22] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length LLM inference with KV cache quantization. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[23] Azam Ikram, Xiang Li, Sameh Elnikety, and Saurabh Bagchi. Ascendra: Dynamic request prioritization for efficient LLM serving. *CoRR*, abs/2504.20828, 2025.

[24] Inferact. Codex agent traces on SWE-Bench Pro. Hugging Face dataset, https://huggingface.co/datasets/Inferact/codex_swebenchpro_traces, 2026.

[25] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. Pod-attention: Unlocking full prefill-decode overlap for faster LLM inference. In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 897–912. ACM, 2025.

[26] Hao Kang, Srikant Bharadwaj, James Hensman, Tushar Krishna, Victor Rühle, and Saravan Rajmohan. Turboattention: Efficient attention approximation for high throughputs llm. In Matei Zaharia, Gauri Joshi, and Yingyan (Celine) Lin, editors, *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025.

[27] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. Aqua: Network-accelerated memory offloading for LLMs in scale-up GPU domains. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 48–62. ACM, 2025.

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023.

[29] Ruihao Li, Shagnik Pal, Vineeth Narayan Pullu, Prasoon Sinha, Jeeho Ryoo, Lizy K. John, and Neeraja J. Yadwadkar. MIRAGE: KV cache optimization through parameter remapping for multi-tenant LLM serving. *CoRR*, abs/2507.11507, 2025.

[30] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: LLM knows what you are looking for before generation. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.

[31] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Yi-Hsiang Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. Adaserve: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, pages 36–54. ACM, 2026.

[32] Dimitrios Liakopoulos, Tianrui Hu, Prasoon Sinha, and Neeraja J. Yadwadkar. iserve: An intent-based serving system for llms. *CoRR*, abs/2501.13111, 2025.

[33] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable. In Ada Gavrilovska and Douglas B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 929–945. USENIX Association, 2024.

[34] Jiachen Liu, Zhiyu Wu, Jae-Won Chung, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing quality-of-experience in LLM-based text streaming services. *CoRR*, abs/2404.16283, 2024.

[35] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*, pages 38–56. ACM, 2024.

[36] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 32332–32344. PMLR / OpenReview.net, 2024.

[37] llm-d. https://github.com/llm-d/llm-d, 2026.

[38] Lmcache: The best kv cache layer for enterprise-scale llm inference. https://github.com/LMCache/LMCache, 2026.

[39] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for LLM agents as general programs. *CoRR*, abs/2502.13965, 2025.

[40] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous GPUs and network via max-flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, pages 586–602. ACM, 2025.

[41] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotserve: Serving generative large language models on preemptible instances. In Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir, editors, *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, pages 1112–1127. ACM, 2024.

[42] MiniMax. MiniMax-M3. https://www.minimax.io/blog/minimax-m3, 2026.

[43] Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Trans. Parallel Distributed Syst.*, 12(10):1094–1104, 2001.

[44] Moonshot AI. Kimi introduces agent swarm: Let 100 AI agents work for you. https://www.kimi.com/blog/agent-swarm, 2026.

[45] NVIDIA. ai-dynamo. https://github.com/ai-dynamo/dynamo, 2026.

[46] OpenAI. OpenAI API. https://openai.com/api/, 2026.

[47] OpenAI. OpenAI API docs: Conversation state. https://developers.openai.com/api/docs/guides/conversation-state, 2026.

[48] OpenClaw. Openclaw. https://openclaw.ai, 2026.

[49] Bei Ouyang, Yifan Qiao, and Jiarong Xing. Unicache: Unifying prefix cache eviction for heterogeneous LLM serving workloads. *Proc. ACM Meas. Anal. Comput. Syst.*, 10(2):54:1–54:27, 2026.

[50] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*, pages 118–132. IEEE, 2024.

[51] Olga Pearce, Todd Gamblin, Bronis R. de Supinski, Martin Schulz, and Nancy M. Amato. Quantifying the effectiveness of load balance algorithms. In Utpal Banerjee, Kyle A. Gallivan, Gianfranco Bilardi, and Manolis Katevenis, editors, *International Conference on Supercomputing, ICS'12, Venice, Italy, June 25-29, 2012*, pages 185–194. ACM, 2012.

[52] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. vattention: Dynamic memory management for serving llms without pagedattention. In Lieven Eeckhout, Georgios Smaragdakis, Kaitai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, ASPLOS 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, pages 1133–1150. ACM, 2025.

[53] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025. USENIX Association.

[54] Qwen Team. Qwen3.7. https://qwen.ai/blog?id=qwen3.7, 2026.

[55] Chaoyi Ruan, Yinhe Chen, Dongqi Tian, Yandong Shi, Yongji Wu, Jialin Li, and Cheng Li. Dynaserve: Unified and elastic tandem-style execution for dynamic disaggregated LLM serving. *CoRR*, abs/2504.09285, 2025.

[56] Sultan Mahmud Sajal, Timothy Zhu, Bhuvan Urgaonkar, and Siddhartha Sen. Traceupscaler: Upscaling traces to evaluate systems at high load. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*, pages 942–961. ACM, 2024.

[57] Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. Don't stop me now: Embedding based scheduling for LLMs. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[58] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 965–988. USENIX Association, 2024.

[59] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. Preble: Efficient distributed prompt scheduling for LLM serving. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.

[60] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In Ada Gavrilovska and Douglas B. Terry, editors, *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 173–191. USENIX Association, 2024.

[61] vllm v0.12.0 release. https://github.com/vllm-project/vllm/releases/tag/v0.12.0, 2025.

[62] Chao Wang, Pengfei Zuo, Zhangyu Chen, Yunkai Liang, Zhou Yu, and Ming-Chang Yang. Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized LLM serving. *CoRR*, abs/2508.01989, 2025.

[63] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, July 2025.

[64] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In Emmett Witchel, Christopher J. Rossbach, Andrea C. Arpaci-Dusseau, and Kimberly Keeton, editors, *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP 2024, Austin, TX, USA, November 4-6, 2024*, pages 640–654. ACM, 2024.

[65] Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. Skywalker: A locality-aware cross-region load balancer for LLM inference. In *Proceedings of the 21st European Conference on Computer Systems, EuroSys 2026, McEwan Hall/The University of Edinburgh, Edinburgh, Scotland, UK, April 27-30, 2026*, pages 1349–1364. ACM, 2026.

[66] Xingyu Xiang, Raj Joshi, Yuhan Liu, Jiayi Yao, Chenxingyu Zhao, Junchen Jiang, Yang Zhou, Eddie Kohler, and Minlan Yu. Shadowserve: Interference-free KV cache fetching for distributed prefix caching. *CoRR*, abs/2509.16857, 2025.

[67] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In Youjip Won, Youngjin Kwon, Ding Yuan, and Rebecca Isaacs, editors, *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP 2025, Lotte Hotel World, Seoul, Republic of Korea, October 13-16, 2025*, pages 1030–1045. ACM, 2025.

[68] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. Servegen: Workload characterization and generation of large language model serving in production. In *23rd USENIX Symposium on Networked Systems Design and Implementation, NSDI 2026, Renton, WA, May 4-6, 2026*, pages 1845–1859. USENIX Association, 2026.

[69] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, pages 94–109. ACM, 2025.

[70] Lu Ye, Ze Tao, Yong Huang, and Yang Li. Chunkattention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 11608–11620. Association for Computational Linguistics, 2024.

[71] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for LLM inference serving. In Matei Zaharia, Gauri Joshi, and Yingyan (Celine) Lin, editors, *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025.

[72] Lingfan Yu and Jinyang Li. Stateful large language model serving with pensieve. *CoRR*, abs/2312.05516, 2023.

[73] Shan Yu, Junyi Shu, Yuanjiang Ni, Kun Qian, Xue Li, Yang Wang, Jinyuan Zhang, Ziyi Xu, Shuo Yang, Lingjun Zhu, Ennan Zhai, Qingda Lu, Jiarong Xing, Youyou Lu, Xin Jin, Xuanzhe Liu, and Harry Xu. Pythia: Exploiting workflow predictability for efficient agent-native LLM serving. *CoRR*, abs/2604.25899, 2026.

[74] Ying Yuan, Pengfei Zuo, Bo Wang, Zhangyu Chen, Zhipeng Tan, and Zhou Yu. Dualmap: Enabling both cache affinity and load balancing for distributed LLM serving. *CoRR*, abs/2602.06502, 2026.

[75] Dingyan Zhang, Jinbo Han, Kaixi Zhang, Xingda Wei, Sijie Shen, Chenguang Fang, Wenyuan Yu, Jingren Zhou, and Rong Chen. Simple is better: Multiplication may be all you need for llm request scheduling. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*, Seattle, WA, USA, July 2026. USENIX Association.

[76] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. Blitzscale: Fast and live large model autoscaling with O(1) host caching. In Lidong Zhou and Yuanyuan Zhou, editors, *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025*, pages 275–293. USENIX Association, 2025.

[77] Rui Zhang, Chaeeun Kim, Shaoting Feng, Kuntai Du, Yuhan Liu, Yi Zhong, Cheng-Wei Ching, Junchen Jiang, and Liting Hu. Learning agent execution for KV-cache management in agentic serving. *CoRR*, abs/2608.14624, 2026.

[78] Yanqi Zhang, Yuwei Hu, Runyuan Zhao, John C. S. Lui, and Haibo Chen. Unifying KV cache compression for large language models with leankv. *CoRR*, abs/2412.03131, 2024.

[79] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Efficiently programming large language models using sglang. *CoRR*, abs/2312.07104, 2023.

[80] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 193–210. USENIX Association, 2024.

[81] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Ziren Wang, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. Nanoflow: Towards optimal large language model serving throughput. In Lidong Zhou and Yuanyuan Zhou, editors, *19th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2025, Boston, MA, USA, July 7-9, 2025*, pages 749–765. USENIX Association, 2025.

[82] Kan Zhu, Haiyang Shi, Le Xu, Jiaxin Shan, Arvind Krishnamurthy, Baris Kasikci, and Liguang Xie. Polyserve: Efficient multi-slo serving at scale. *CoRR*, abs/2507.17769, 2025.

```
req = receive()                          (a) stateless session-centric routing
c = kvc.hit_len(req)    # local KV$ hit, per instance
q = queued_cost()  # sum of prefill_cost in i's queue
prev = instances.argmax(c)  # likely served last turn
cand = [i for i in instances if meets_tpot(req, i)]
if not cand: cand = instances  # (b) none passes

if req.turn != 0 and session_cached(req, prev) \
        and meets_ttft(req, prev) \
        and prev in cand:                    # (b)
    sched_to = prev                          # stick
else:                                        # balance
    sched_to = cand.rr_argmin(key=lambda i:
        estimate_load(req, i) * i.num_requests)
req.forward(sched_to)

def session_cached(req, i): # i caches req's session
    return c[i] > HIT_RATIO * est_hit(req)
    # est_hit: the hit expected from req's history
```

```
def meets_ttft(req, i):                  (b) SLO checks
    # the deadline check
    ttft = estimate_load(req, i) / PREFILL_RATE
    return ttft <= SLACK * ttft_slo(req)

def meets_tpot(req, i):
    # i's live decodes stay within their TPOT SLO
    stall = prefill_cost(req, c[i]) / PREFILL_RATE
    return all(stall <= slo_slack(d)
               for d in i.live_decodes())
```

```
def estimate_load(req, i):               (c) load model
    # i's queued prefill + req's own
    return q[i] + prefill_cost(req, c[i])

def prefill_cost(req, hit): # Eq. 1, reusing hit
    L = req.prompt_len; n = L - hit  # new tokens
    return C_LIN * n + C_ATT * n * (L - n / 2)
```

Figure 27: Pseudocode of SMETRIC under PD colocation. The bars (|) mark the lines that differ from Figure 13. The constants and hyperparameters are the same as Figure 13.

A Session reconstruction from anonymized trace

As the requests collected in our traces have no session identifier, we reconstruct sessions offline to simplify analysis and to compare with session ID-based scheduling methods.

Specifically, each request in our trace contains its arrival time, an opaque user ID (a user may be one person or a whole organization), the prompt as a sequence of 512-token block hashes (the trace carries no prompt text), the input and output lengths, the turn number within its session, and what triggers the request, e.g., whether the request is a compaction request or a normal one sent by the user or the agent.

To decide which session a request belongs to, the observation is that one request is the follow up of another from the same user if: (1) its turn is larger than the other by 1 and (2) its hashed block sequence starts with that of the other, because each turn extends the prompt of its parent with the model's reply and the tool output or the next user prompt. As a result, we can simply link each request to the earlier request of the same user that satisfies both conditions, chain requests together, and finally assign requests in the same chain the same session ID.

B SMETRIC under PD colocation

Figure 27 presents the pseudocode of SMETRIC under PD colocation. Its high-level structure is similar to the PD-disaggregated version in Figure 13, with two main PD-colocation-aware changes.

First, we further select the instances that meet the TPOT SLO (lines 5 and 9) before deciding where to route, because a colocated instance also executes decodes, which the prefill of the new request delays. If no instance meets the TPOT SLO, the check is skipped (line 6). Note that the check only considers the decodes in flight at the instance, because the number of tokens the new request will decode is unknown. Second, we adopt the multiplicative score of LMetric [75] to select the target of load balancing and session migration (lines 12–13), because in our setup it is the most effective score that considers prefill and decode in a unified way.

C SLO sensitivity on the larger model

Figure 28 reports the sensitivity to the SLO on Qwen3-235B (§5.2). SMETRIC achieves the best performance at various SLO setups: by 8–16 % across the fixed and per-token TTFT budgets and by 4.5–24 % across TPOT SLOs of 20–67 ms.

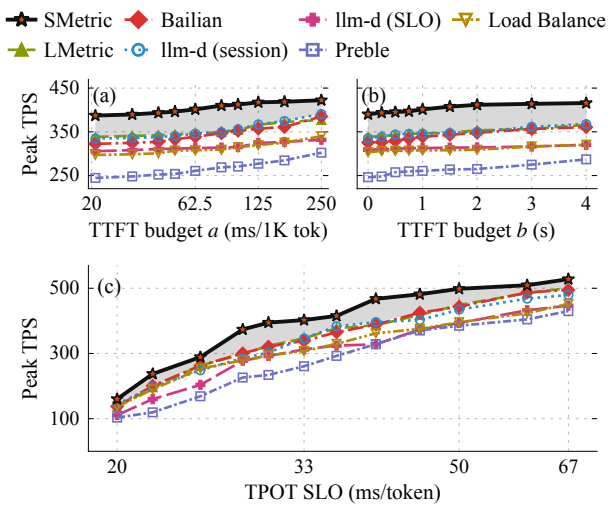


Figure 28: Sensitivity to SLO configurations on the 8-instance Qwen3-235B-A22B-FP8 PD-colocated setting.

D Results on an open agentic-coding trace

To confirm that the PD-disaggregation results (§5.3) are not specific to our trace, we repeat them on an open trace, the Codex agent traces on SWE-Bench Pro [24], which record 610 sessions (20,230 requests) of the Codex coding agent. The trace carries every prompt and response length but no timestamps, so we sample the think time between turns from the inter-call delays reported with the dataset. Its sessions are longer than ours (33 turns on average vs. 2.3) and reuse more of their prompts (94 % of the prompt tokens are cache hits, vs. about 80 % in ours). The setup is otherwise that of §5.3.

The results are consistent with §5.3. SMETRIC achieves the highest peak prefill TPS, 10 % above Dynamo, the best baseline (Figure 29 (a)), and the most balanced load after LMetric, which gives up KV$ reuse for balance (Figure 29 (b)): the busiest instance holds 2.5× as many requests as the average instance, against 3.1× or more under the other baselines. It matches or exceeds the best baseline at every global-tier capacity (Figure 30) and across TTFT budgets (Figure 32), and delivers the lowest TTFT at both the median and the tail (Figure 31). LMetric and BAILIAN collapse at a lower request rate than on our trace, for different reasons: LMetric serves most follow-ups from the global tier (Figure 30 (b)), and the longer prompts make each such transfer costlier; BAILIAN weighs KV$ hits far above load, so it keeps sending a session's follow-ups to its home even when the home is busy, and the long sessions keep it busy.

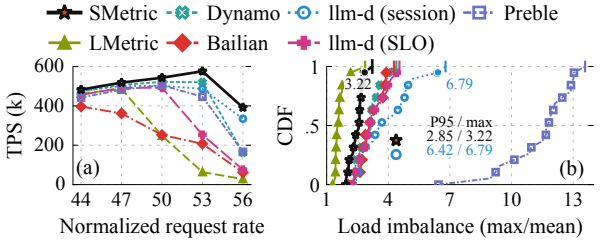


Figure 29: (a) Prefill TPS within SLO and (b) distribution of the per-window load imbalance, as in Figure 21, on the 32-instance Qwen3-30B-A3B PD-disaggregated setting, replaying the Codex trace.

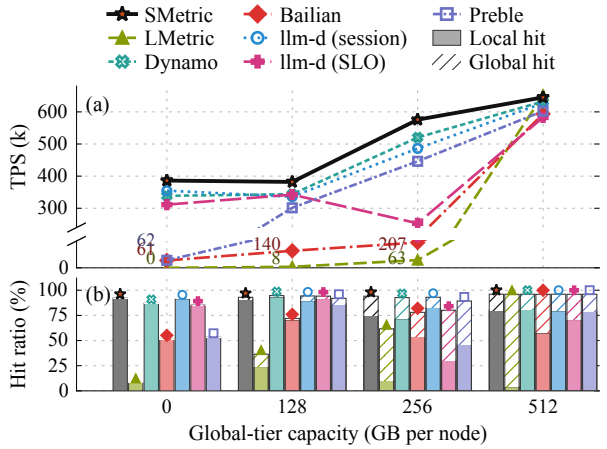


Figure 30: (a) Prefill TPS within SLO and (b) KV$ reuse ratio and its source under different global-tier capacities on the 32-instance Qwen3-30B-A3B PD-disaggregated setting, replaying the Codex trace.

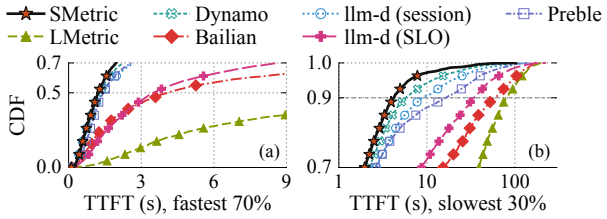


Figure 31: TTFT distributions on the 32-instance Qwen3-30B-A3B PD-disaggregated setting for (a) common and (b) tail requests, replaying the Codex trace. (b) uses a log scale.

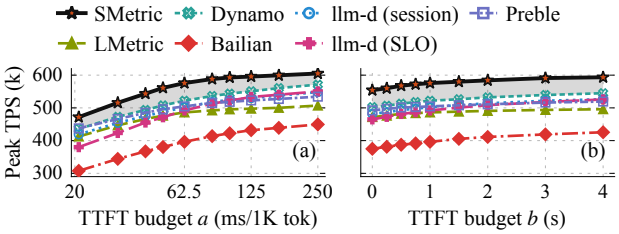


Figure 32: Sensitivity to SLO configurations on the 32-instance Qwen3-30B-A3B PD-disaggregated setting, replaying the Codex trace.